UCS 503 Software Engineering Project Report

Mid-Semester Evaluation

Government Exam Notification and Deadline Alert System

Submitted by:

1024030731 – Aryan Juneja

1024030747 – Raghav Mittal

1024031150 – Bir Angad

Group: 3C53

BE Third Year, COE

Submitted to:

PARAMVEER KAUR

Computer Science and Engineering Department

Thapar Institute of Engineering and Technology, Patiala

Submission Date: 16 October 2026

Contents

1 Introduction **1**

1.1 Background and Context 1

1.1.1 Problem Statement 1

1.2 Project Objectives 1

1.3 Project Scope 2

2 Methodology and Design **3**

2.1 System Architecture 3

2.1.1 High-Level Design 3

2.1.2 Technical Stack 4

2.2 Implementation Details 4

2.2.1 Module 1: Exam Listing Interface 4

2.2.2 Module 2: Backend API and Data Management 5

2.2.3 Planned Module: Deadline Reminders 5

2.3 Analysis Phase: Requirements and Models 5

2.3.1 Actors and Use-Case Diagram 5

2.3.2 Use-Case Templates 6

2.3.3 Activity Diagram 7

2.3.4 Swimlane Workflow 8

2.3.5 Data Flow Diagrams 9

2.3.6 Sequence Diagram 10

2.3.7 Data Model 10

2.4 Software Requirements Specification (IEEE-Style) 11

2.4.1 Purpose and Scope 11
2.4.2 User Classes and Operating Environment 11
2.4.3 Functional Requirements 11
2.4.4 Non-Functional Requirements 12
2.4.5 Data Requirements and Constraints 12
2.4.6 Assumptions and Dependencies 13
2.5 User Stories and Story Cards 13
2.5.1 Requirement Traceability 14

3 Results and Evaluation 16
3.1 Testing Methodology 16
3.2 Performance Metrics 16
3.3 Analysis 17

4 Conclusion 18
4.1 Summary of Work 18
4.2 Key Findings 18
4.3 Future Work and Recommendations 18
4.4 Final Remarks 19

Chapter 1

Introduction

1.1 Background and Context

Government recruitment examinations are announced by multiple organisations, each publishing information such as eligibility, application dates, examination dates, and official application links. Aspirants often need to visit several websites and manually track deadlines. This can make it difficult to compare opportunities and may cause applicants to miss important dates.

GovExam Alert is a web-based project intended to organise government examination information in one accessible portal. Its interface presents exam details in a structured form, while a backend API is being developed to retrieve and manage exam records. Deadline reminders and complete persistent data integration are planned extensions of the current prototype.

1.1.1 Problem Statement

Exam-related information is spread across multiple official sources and may be updated at different times. Users need a straightforward way to find relevant examinations and review their key dates and application links. The project addresses this need through a centralised listing interface and a planned data-management workflow. Official notifications remain the authoritative source for dates, eligibility, and application instructions.

1.2 Project Objectives

The objectives of the project are:

1. Build a clear web interface that displays government examination listings and their key details.
2. Develop backend API routes to retrieve exam records and support administrative creation, updating, and deletion.
3. Connect the frontend to backend data and verify that records can be stored and retrieved reliably.
4. Plan and implement deadline reminders after the core data flow is stable.
5. Test the system's main user flows and API operations, documenting observed results and unresolved issues.

This report documents the work completed so far and clearly identifies unfinished items. The frontend prototype and parts of the exam-management API have been developed, but backend reliability, complete frontend–backend integration, database persistence verification, and deadline-reminder delivery remain unfinished or require retesting. The design models show the intended project behaviour; they do not mean that every feature is operational.

1.3 Project Scope

The current scope focuses on displaying exam information and building a basic API for exam records. The wider intended scope includes persistent data management, search and category filtering, administrative exam management, and deadline reminders. The project is an academic prototype and should not be treated as an official source of recruitment information or as a production-ready notification service.

Chapter 2

Methodology and Design

2.1 System Architecture

GovExam Alert is intended to use a client–server design. The React frontend presents exam listings and user interactions; the Express backend exposes routes for reading and managing exam records; and SQLite is the planned local data store. These parts are not yet verified as a reliable end-to-end integration. The evaluation chapter records the tests actually observed and identifies unfinished integration and data-persistence checks.

2.1.1 High-Level Design

Figure 2.1 presents the intended components. The reminder component is planned and the end-to-end connections still require verification.

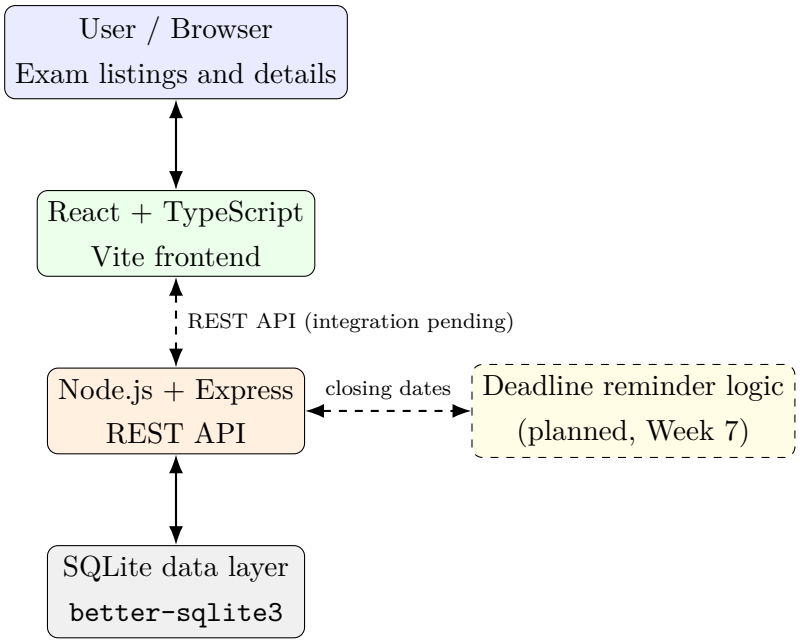


Figure 2.1: High-level architecture of GovExam Alert. Dashed elements and links indicate planned or not-yet-verified functionality.

2.1.2 Technical Stack

Component	**Technology / Tool**
Frontend	React with TypeScript
Build and development server	Vite
Routing and icons	React Router and Lucide React
Backend	Node.js and Express
Database layer	SQLite with `better-sqlite3`
API testing	Thunder Client
Development environment	Visual Studio Code and npm
Version control	Git and GitHub

2.2 Implementation Details

2.2.1 Module 1: Exam Listing Interface

The frontend prototype includes a landing page for GovExam Alert and exam cards. The exam data model contains fields such as exam title, recruiting organisation, category,

application last date, exam date, application link, description, eligibility information, number of posts, and an indicator for new listings. Sample records include SSC CGL, UPSC Civil Services, and other illustrative examination entries in the frontend prototype.

The frontend currently demonstrates the presentation and browsing experience. Because some screens use mock data, the visible listings should not be interpreted as proof that every record is being loaded from the backend or verified against a live official notification.

2.2.2 Module 2: Backend API and Data Management

The Express backend exposes a basic health route and exam-related endpoints, including routes to retrieve exam records and administrative routes for creating, updating, and deleting records. Search and category-filtering functionality is part of the API work. SQLite with `better-sqlite3` has been installed for local data storage.

Previously recorded Thunder Client tests showed a successful POST response with HTTP status 201 and a successful PUT response with HTTP status 200. A DELETE request returned HTTP 404 during testing and remains unresolved. At the latest reporting point, backend API response-time measurements have not been collected because the API is not currently working reliably. These operations must be retested after the issue is fixed.

2.2.3 Planned Module: Deadline Reminders

The Week 7 design includes a deadline-reminder workflow based on application closing dates. Reminder intervals, delivery channels, scheduling logic, and user preferences should be validated against the final implementation before the feature is considered operational.

2.3 Analysis Phase: Requirements and Models

This section expands the design documentation using the analysis deliverables represented in the two reference reports. The diagrams model the intended GovExam Alert workflow; they do not imply that every depicted feature has already been implemented. In particular, reminders and complete frontend–backend integration remain planned work.

2.3.1 Actors and Use-Case Diagram

The primary actor is the aspirant/user who browses exam listings, searches by category or keyword, views details and follows official application links. An administrator maintains

examination records through create, update and delete operations. The backend API and database support these interactions. Deadline reminders are a planned use case.

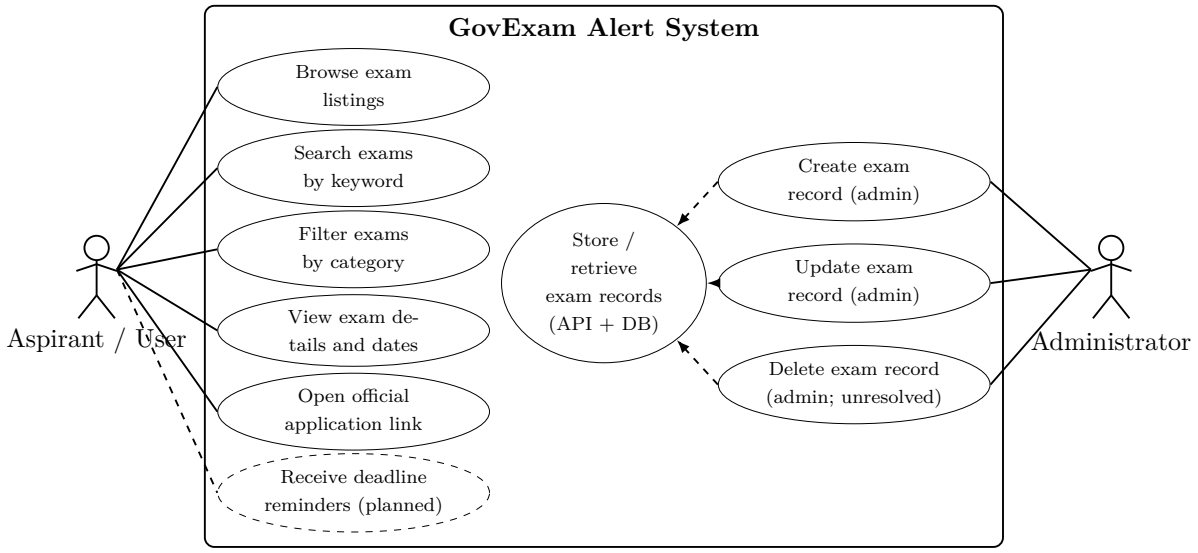


Figure 2.2: Use-case diagram for GovExam Alert. Dashed ellipse = planned feature; dashed arrows = «include» relationships. The delete operation has a recorded HTTP 404 failure.

2.3.2 Use-Case Templates

UC-01: Browse Exam Listings

Actor: Aspirant/user. **Precondition:** The application is open.

Main flow: (1) User opens the listing page. (2) The system displays available exam cards. (3) User browses the cards and selects an exam. (4) The system displays available details.

Alternative flow: If no records are available, the interface should show an empty state or an error message.

Postcondition: The user can view the available exam information.

UC-02: Search or Filter Exams

Actor: Aspirant/user. **Precondition:** Exam listings are available.

Main flow: (1) User enters a keyword or selects a category. (2) The system applies the search/filter condition. (3) Matching records are displayed.

Alternative flow: If there are no matches, show a clear no-results message.

Postcondition: A filtered list or a no-results state is displayed.

UC-03: Manage Exam Record

Actor: Administrator. **Precondition:** The admin endpoint is available and the request contains a valid record.

Main flow: (1) Admin submits a create or update request. (2) API validates the request. (3) The record is stored or updated. (4) API returns a success response.

Alternative flow: Invalid data or a missing record returns an appropriate error. Delete behaviour requires further testing because a recorded DELETE request returned HTTP 404.

Postcondition: The operation result is returned; persistence must be verified through regression tests.

UC-04: Follow Official Application Link

Actor: Aspirant/user. **Precondition:** An exam record includes an application URL.

Main flow: User opens exam details and selects the official application link; the browser navigates to the supplied destination.

Alternative flow: If the URL is missing or unavailable, the interface should tell the user to consult the recruiting organisation's official website.

Postcondition: User is directed to the provided official source.

UC-05: Receive Deadline Reminder (Planned)

Actor: Aspirant/user; supporting component: reminder scheduler. **Precondition:** Reminder logic and delivery method have been implemented and configured.

Main flow: The scheduler checks upcoming closing dates and sends reminders according to configured intervals.

Alternative flow: Missing dates or failed delivery are logged and must not be represented as a successful reminder.

Postcondition: A reminder is delivered only when the planned service is operational. This use case belongs to the Week 7 scope; successful delivery still requires verification.

2.3.3 Activity Diagram

Figure 2.3 describes the user browsing flow and the planned reminder branch.

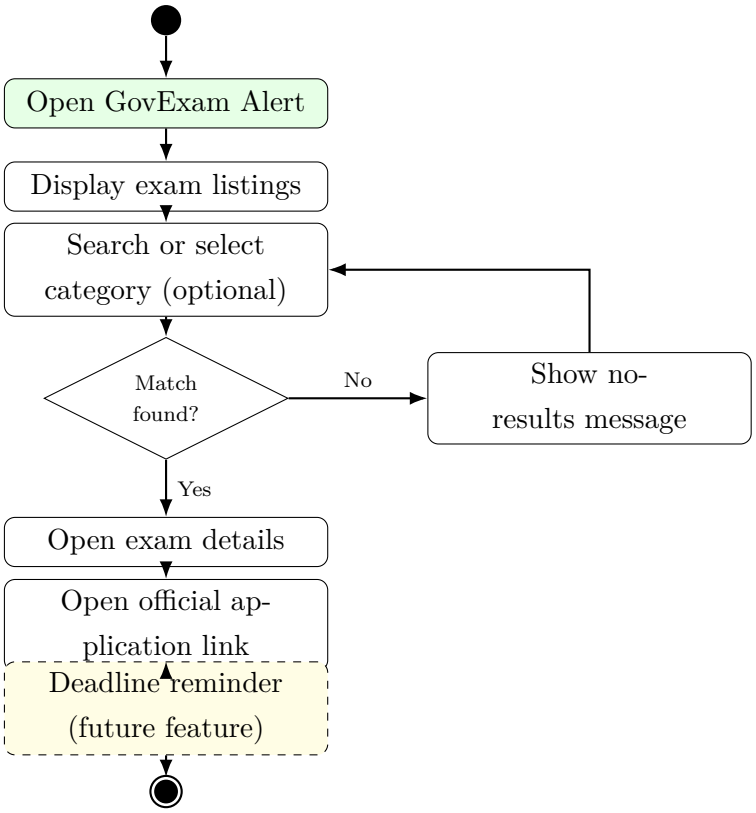


Figure 2.3: Activity diagram of the user browsing workflow. The no-results branch returns to search; the reminder step is planned.

2.3.4 Swimlane Workflow

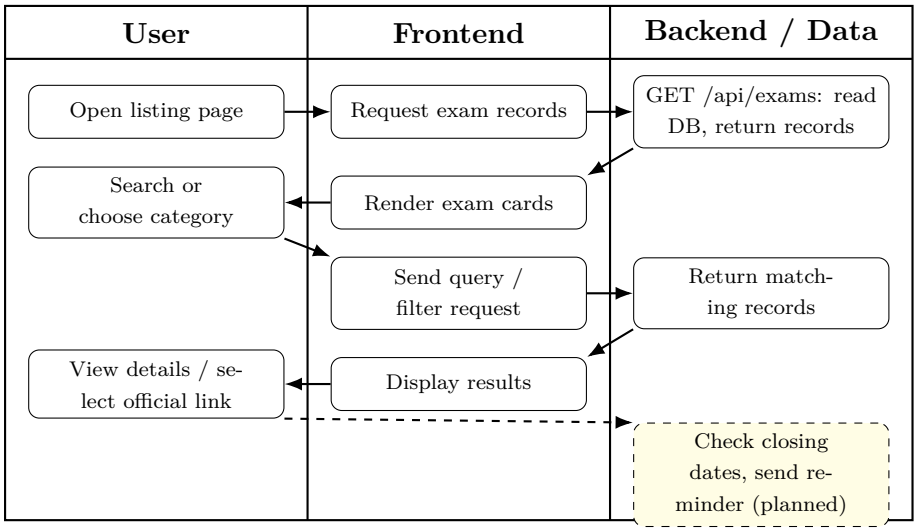


Figure 2.4: Swimlane view of the user, frontend, and backend responsibilities. All backend data reaches the user through the frontend.

2.3.5 Data Flow Diagrams

DFD Level 0: Context Diagram

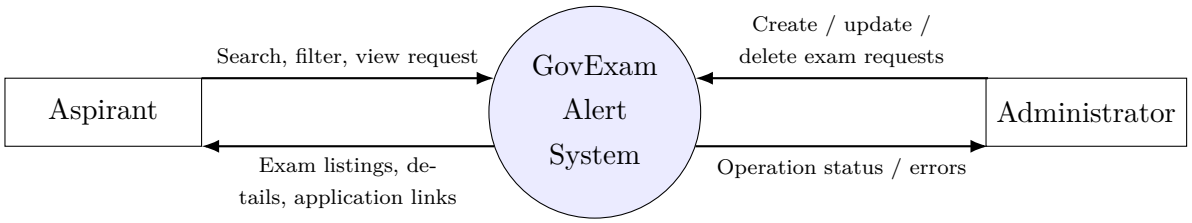


Figure 2.5: DFD Level 0 showing external actors and the system boundary.

DFD Level 1: Main Processes

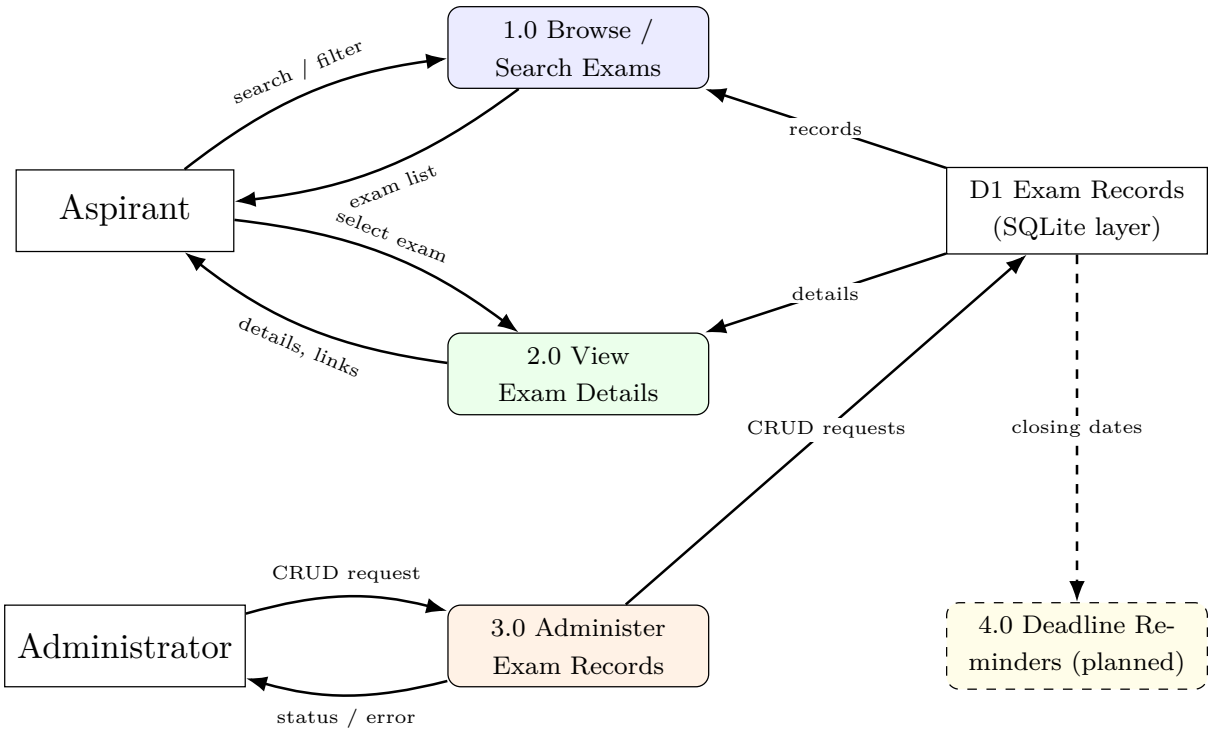


Figure 2.6: DFD Level 1 showing external entities, browsing, details, administration, and the planned reminder process.

DFD Level 2: Exam Administration

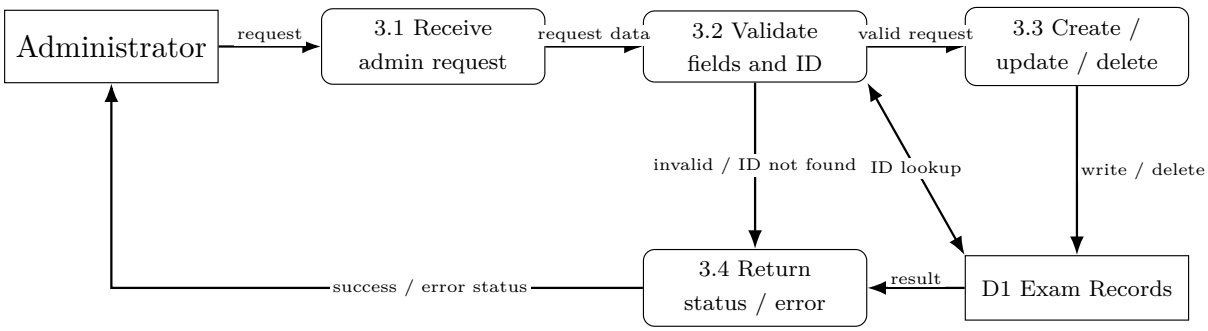


Figure 2.7: DFD Level 2 expanding administrative exam-record processing, including the ID existence check against the data store.

2.3.6 Sequence Diagram

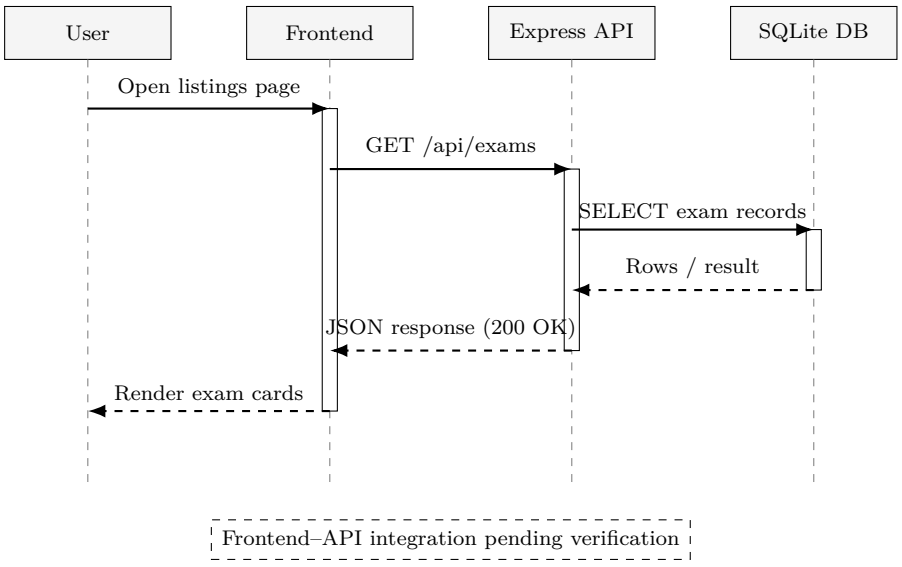


Figure 2.8: Intended sequence for retrieving exam listings (solid = request, dashed = response); current frontend integration remains pending.

2.3.7 Data Model

The planned core exam record contains an identifier, title, recruiting organisation, category, application closing date, exam date, official application URL, description, eligibility details, and number of posts. Additional fields may be added only when needed and supported by the source notification. Database persistence and frontend consumption must be verified through testing.

2.4 Software Requirements Specification (IEEE-Style)

This concise SRS follows an IEEE-style organisation for a course project. It documents intended requirements and is not a claim that all requirements are implemented.

2.4.1 Purpose and Scope

GovExam Alert aims to help aspirants browse government exam information in one portal. The baseline scope includes listing exam records, viewing details, searching/filtering records, and maintaining records through administrative API operations. Reminder delivery is a planned extension.

2.4.2 User Classes and Operating Environment

- **Aspirant/user:** browses, searches, filters, views details and follows official application links.
- **Administrator:** creates, updates and deletes exam records through administrative endpoints.
- **Operating environment:** modern desktop browser; React/TypeScript frontend; Node.js/Express backend; local SQLite data layer.

2.4.3 Functional Requirements

ID	Requirement
FR-01	The system shall display a list of available examination records.
FR-02	The system shall display key exam details, including title, organisation, category, dates, description and application link when available.
FR-03	The system shall support searching or filtering examination records where the corresponding endpoint/interface is available.
FR-04	The system shall provide an official application URL when one is recorded.
FR-05	The backend shall expose an endpoint to retrieve examination records.
FR-06	The backend shall support creating a valid exam record through an administrative endpoint.

FR-07	The backend shall support updating an existing exam record through an administrative endpoint.
FR-08	The backend shall support deleting an exam record; the recorded DELETE failure must be fixed and retested.
FR-09	The system shall validate required fields and return clear success or error responses.
FR-10	The system should provide deadline reminders once scheduling and delivery are implemented.

Table 2.2: Functional requirements for GovExam Alert.

2.4.4 Non-Functional Requirements

ID	Requirement
NFR-01	Usability: exam information should be organised into readable cards or detail views.
NFR-02	Reliability: API errors should be reported clearly and should not silently appear as valid data.
NFR-03	Data integrity: required exam fields and record identifiers should be validated.
NFR-04	Maintainability: frontend presentation, API logic and data access should remain separated.
NFR-05	Performance: page and API response times should be measured under documented conditions; no API target is claimed as achieved yet.
NFR-06	Accuracy: dates, eligibility and URLs should be checked against official recruiting notifications before being presented as verified current information.
NFR-07	Compatibility: the frontend should operate in a current mainstream browser.

Table 2.3: Non-functional requirements.

2.4.5 Data Requirements and Constraints

An exam record should contain an ID, title, organisation, category, last application date, exam date, official application link and description where available. Eligibility and number of posts may be included when available in the source. Dates and application details

can change; official recruiting notifications remain authoritative. The backend's current DELETE failure and incomplete frontend integration are known implementation constraints.

2.4.6 Assumptions and Dependencies

- Users have access to a modern browser and an internet connection or the local development environment.
- Official recruitment organisations publish the authoritative exam details.
- Backend routes and database connectivity must be available for live data to be displayed.
- Reminder functionality depends on a reliable source of closing dates and a configured delivery/scheduling mechanism.

2.5 User Stories and Story Cards

User stories state the expected value from the user's perspective. Acceptance criteria are proposed verification conditions; they do not indicate that a feature has passed testing.

ID	Story	Acceptance criteria	Priority
US-01	As an aspirant, I want to browse exam listings so that I can discover opportunities.	Listings display available exam titles and basic details; empty/error states are clear.	High
US-02	As an aspirant, I want to search or filter exams so that I can narrow results.	Query/category filters return matching records and show a no-results state.	High
US-03	As an aspirant, I want to open exam details so that I can review dates and eligibility.	Detail view shows available fields and does not invent missing values.	High
US-04	As an aspirant, I want an official application link so that I can apply at the source.	Link points to the stored official URL and is clearly labelled.	High

US-05	As an administrator, I want to add an exam so that new notifications can be listed.	Valid POST creates a record; invalid data returns an error.	High
US-06	As an administrator, I want to update an exam so that corrected details can be stored.	Valid PUT updates the correct record and the change can be retrieved.	High
US-07	As an administrator, I want to delete an obsolete record so that the listing stays current.	DELETE succeeds for an existing record and returns a clear not-found response for a missing ID.	High
US-08	As an aspirant, I want reminders before a deadline so that I can avoid missing closing dates.	Configured reminders arrive at the chosen intervals; this feature is planned and not yet implemented.	Medium

Table 2.4: User-story summary and acceptance criteria.

2.5.1 Requirement Traceability

Requirements	**Related stories**	**Verification approach**
FR-01–FR-04	US-01–US-04	Browser walkthrough; verify listing fields and links.
FR-05	US-01–US-03	GET request and response-body validation.
FR-06	US-05	POST valid and invalid payloads; verify persisted record.
FR-07	US-06	PUT update and subsequent GET.
FR-08	US-07	DELETE existing and missing IDs; fix the recorded 404 issue.
FR-10	US-08	Future scheduled reminder test after implementation.

NFR-01–NFR-07	All relevant stories	Usability checks, input validation, compatibility checks, and repeatable performance tests.

Table 2.5: Traceability between requirements, user stories, and verification.

Chapter 3

Results and Evaluation

3.1 Testing Methodology

Testing to date has primarily involved manual browser checks and API requests through Thunder Client. The recorded checks include:

- Opening the frontend in a browser to confirm that the interface loads.
- Retrieving exam records from the API in earlier tests.
- Sending a POST request to create an exam; the recorded response was HTTP 201 Created.
- Sending a PUT request to update an exam; the recorded response was HTTP 200 OK.
- Sending a DELETE request; the recorded response was HTTP 404 Not Found.

Automated unit tests, comprehensive integration tests, user testing, and repeatable performance benchmarking have not yet been completed. The DELETE failure and current backend availability issue remain open items.

3.2 Performance Metrics

Frontend page-load measurements were collected in three browser attempts. The observed values were 564 ms, 570 ms, and 528 ms. The arithmetic mean is:

$$\bar{t} = \frac{564 + 570 + 528}{3} = 554 \text{ ms}.$$

This is the average of the supplied page-load observations, not a guarantee of performance on other devices or networks. No valid backend API latency measurements were supplied, so the API response-time metric remains pending.

Table 3.1: Mid-Semester Performance and Test Summary

Metric / Operation	**Observed Result**	**Status**
Frontend page-load time	564, 570, 528 ms; average 554 ms	Measured
GET /api/exams response time	No valid readings available	Pending
Create exam (POST)	HTTP 201 Created in recorded test	Previously passed; retest required
Update exam (PUT)	HTTP 200 OK in recorded test	Previously passed; retest required
Delete exam (DELETE)	HTTP 404 Not Found in recorded test	Unresolved
Frontend loads in browser	Interface opens	Basic check completed
Frontend–backend integration	Week 7 integration scope	Verification pending
Deadline-reminder workflow	Included in Week 7 design scope	Validation pending

3.3 Analysis

The project remains unfinished. The three frontend page-load observations average 554 ms in the recorded environment. Earlier POST and PUT checks returned success status codes, showing that those particular requests were accepted at the time of testing.

The recorded test evidence is narrower than the full Week 7 project scope: the DELETE route returned 404 in a recorded test, and backend API latency could not be measured. These are validation issues, not fabricated successes. Reminder delivery and the full frontend-to-backend path still require verification. The next evaluation should repeat the API tests after restoring the backend and should measure API latency over several successful requests.

Chapter 4

Conclusion

4.1 Summary of Work

GovExam Alert is being developed as a centralised portal for government examination information. Work to date includes a React/TypeScript frontend prototype, an Express backend with exam-related routes, initial local SQLite setup, and manual API testing. The frontend page-load observations supplied for this report average 554 ms. Backend response-time measurements remain unavailable.

4.2 Key Findings

- The frontend prototype provides a structured way to present exam details and key dates.
- Earlier API tests recorded HTTP 201 for creating an exam and HTTP 200 for updating an exam.
- The DELETE endpoint's recorded 404 response and current API reliability issue need resolution before the API can be considered stable.
- Validation of the full frontend–backend path, persistent data flow, and deadline-reminder delivery remains a priority.

4.3 Future Work and Recommendations

1. Diagnose and fix the backend availability and DELETE endpoint issues.
2. Retest GET, POST, PUT, and DELETE operations with valid and invalid inputs.

3. Connect the frontend to the backend API and reduce reliance on mock data in the integrated flow.
4. Verify persistence by creating, updating, retrieving, and deleting records through the database-backed API.
5. Implement deadline reminders only after the exam data flow is reliable; define reminder intervals and delivery methods.
6. Add automated tests and repeatable performance measurements, recording the environment and test conditions.
7. Validate dates, eligibility, and application links against official recruiting notifications before displaying them as current information.

4.4 Final Remarks

The mid-semester work provides a useful foundation for GovExam Alert, but important integration and reliability tasks remain. The immediate priority is to restore stable backend operation, resolve the DELETE failure, and verify the complete data flow. The final project evaluation should be based on repeatable test results and clearly distinguished implemented features versus planned features.

Bibliography

[1] React Documentation. *React*. Available at: https://react.dev/

[2] Vite Documentation. *Vite*. Available at: https://vite.dev/

[3] Express Documentation. *Express – Node.js web application framework*. Available at: https://expressjs.com/

[4] SQLite Documentation. *SQLite*. Available at: https://www.sqlite.org/docs.html

[5] Thapar Institute of Engineering and Technology. *UCS503 Software Engineering Lab report template*.